

QA METRICS PORTAL

Proyecto:	Plataforma Web de Gobierno y Métricas de QA	Versión:	2.0.0 (Enterprise)
Stack Principal:	Angular 21 + Tailwind CSS + Spring Boot 3.x	Estrategia QA:	JUnit 5, Mockito, Karma, Playwright E2E

1. Visión General del Sistema y Requerimientos

El sistema **QA Metrics Portal** tiene como objetivo sustituir la gestión operativa basada en hojas de cálculo planas por una arquitectura distribuida, segura y automatizada. Permite controlar la ejecución de pruebas, el registro de bugs, la trazabilidad de fechas de entrega y el cálculo automatizado de indicadores de calidad por proyecto (Fábrica vs Minor Demand).

1.1 Requerimientos Funcionales (RF)

- RF01 - Segregación por Línea de Operación:** Clasificación nativa de cada registro en **FÁBRICA** o **MINOR DEMAND** para generación de tableros estadísticos independientes o consolidados.
- RF02 - Gestión de Diseño y Retests Iterativos:** Registro base de requerimiento/Jira con soporte para N iteraciones de retest (fecha, analista ejecutante y resultado) sin límite de columnas.
- RF03 - BugTracker y Análisis de Defectos:** Registro de incidencias clasificadas por Sprint/PI, observaciones de reinyección/regresión y categorización para determinar los requerimientos más defectuosos.
- RF04 - Control de Hitos y Tiempos de Entrega:** Trazabilidad de 3 fechas críticas: Compromiso Estimado, Entrega Real a QA y Entrega Real a Cliente.
- RF05 - Motor de Indicadores y Reportería:** Cálculo automático de ratios de ejecución y del % de Calidad (Target 95%), con generación de reportes quincenales/mensuales en PDF y XLSX.
- RF06 - Control de Acceso basado en Roles (RBAC):** Autenticación JWT Stateless diferenciando entre perfil **ADMIN** y **ANALYST**.

1.2 Requerimientos No Funcionales (RNF)

- RNF01 - Backend Clean Architecture:** Implementación de Arquitectura Hexagonal (Ports & Adapters) en Spring Boot 3.x con Java 21.
- RNF02 - Frontend Reactivo Standalone:** Angular 21 utilizando Signals, estructuras Standalone y Tailwind CSS para un diseño adaptativo de alto rendimiento.
- RNF03 - Estrategia de Testing Integral:** Cobertura unitaria >80% en Backend (JUnit 5 + Mockito) y Frontend (Jasmine/Karma), sumado a suite E2E automatizada con **Playwright**.
- RNF04 - Infraestructura Costo Cero / Low Cost:** Despliegue distribuido en Vercel (Frontend), Render/Railway (Backend) y Neon PostgreSQL (Base de Datos).

2. Stack Tecnológico Estándar

Capa / Dominio	Tecnología Seleccionada	Justificación Técnica
Frontend Framework		

Capa / Dominio	Tecnología Seleccionada	Justificación Técnica
	Angular 21 (Standalone + Signals)	Eliminación de NgModules legados, renderizado reactivo fino mediante Signals.
Estilos CSS	Tailwind CSS v3/v4	Utilidades CSS responsivas, componentes estilizados sin sobrecarga de runtime.
Visualización / Gráficos	Apache ECharts / Ngx-ECharts	Soporte para velocímetros (Gauge Target 95%), gráficos de área comparativos e histogramas.
Backend Framework	Java 21 / Spring Boot 3.3+	Patrón Hexagonal puro, rendimiento Enterprise y excelente ecosistema de seguridad.
Seguridad	Spring Security 6 + JWT Stateless	Cifrado BCrypt, tokens de acceso firmados HMAC256, filtros CORS/CSRF configurados.
Base de Datos	PostgreSQL 16 (Neon / Supabase)	Motor relacional robusto con soporte JSONB si se requiere y capa gratuita ilimitada.
Pruebas Unitarias / Mocks	JUnit 5, Mockito, Jasmine/Karma	Aislamiento de lógica de negocio en backend y componentes aislados en frontend.
Pruebas E2E (End-to-End)	Playwright (Node.js / TypeScript)	Automatización nativa multi-browser, ejecuciones paralelas, autofixtures y reportes HTML.

3. Arquitectura de Software y Modelo de Datos

3.1 Estructura Hexagonal en Spring Boot

```
com.qametrics.portal
├── domain                                <-- Núcleo Puro de Negocio
│   ├── model                            <-- Entidades (TestExecution, ExecutionRun, Bug)
│   ├── port                             <-- Inbound & Outbound Ports (Interfaces)
│   └── service                           <-- Reglas de negocio (% Calidad, SLA, Ratios)
├── application                           <-- Casos de Uso
│   └── usecase                           <-- Orquestación de servicios e interactores
├── infrastructure                        <-- Adaptadores e Implementación Técnica
│   ├── adapters
│   │   ├── in.rest                       <-- Controllers REST (Spring MVC)
│   │   └── out.persistence                <-- Repositorios Spring Data JPA & Mapeadores
│   └── config                             <-- Security JWT, Cors, Beans Configuration
```

3.2 Modelo Relacional Normalizado (Solución al Retest)

Para resolver las columnas múltiples e iterativas del Excel actual (Retest 1, Retest 2, etc.), se implementa una relación 1 a N (One-To-Many) entre el registro de la prueba y sus iteraciones de ejecución:

```
-- Tabla Principal de Pruebas / Requerimientos
CREATE TABLE test_executions (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  jira_id VARCHAR(50) NOT NULL,
  project_type VARCHAR(20) NOT NULL, -- 'FABRICA' o 'MINOR_DEMAND'
  assignment_date DATE NOT NULL,
  design_date DATE,
  designer_analyst VARCHAR(100) NOT NULL,
```

```

        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    );

-- Tabla Hija de Ejecuciones e Iteraciones de Retest (Sustituye columnas horizontales)
CREATE TABLE test_execution_runs (
    id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    test_execution_id BIGINT NOT NULL,
    run_number INT NOT NULL, -- 1 = Ejecución inicial, 2 = Retest 1, 3 = Retest 2...
    execution_date DATE NOT NULL,
    executed_by_analyst VARCHAR(100) NOT NULL,
    status VARCHAR(20) NOT NULL, -- 'SUCCESSFUL', 'FAILED', 'BLOCKED', 'RETEST'
    CONSTRAINT fk_execution FOREIGN KEY (test_execution_id) REFERENCES test_executions(id) ON
    DELETE CASCADE
);

-- Tabla de Registro de Incidencias (BugTracker)
CREATE TABLE bugs (
    id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    bug_jira_id VARCHAR(50) NOT NULL UNIQUE,
    requirement_id VARCHAR(50) NOT NULL,
    sprint_or_pi VARCHAR(50) NOT NULL,
    status VARCHAR(20) NOT NULL,
    defect_type VARCHAR(30) NOT NULL, -- 'DEVELOPMENT_ERROR', 'REINJECTION', etc.
    description TEXT NOT NULL,
    reinjection_flag BOOLEAN DEFAULT FALSE,
    reported_date DATE NOT NULL
);

```

4. Formulas e Indicadores Clave de Calidad

El portal automatiza el cálculo de métricas consolidándolas por Sprint, PI, Mes o Línea de Operación:

$$\text{Ratio de Ejecución} = \frac{\sum \text{Casos Exitosos}}{\sum \text{Total de Intentos de Ejecución}}$$

$$\% \text{ Porcentaje de Calidad} = (1 - (\text{Bugs Encontrados en Periodo} / \text{Casos Exitosos en Periodo})) \times 100$$

Target de Calidad Objetivo: 95%

Si el % de Calidad calculado para un periodo o proyecto es inferior a 95%, el dashboard de Tailwind CSS resaltará el indicador mediante un banner de alerta condicional en rojo/amarillo.

5. Estrategia Completa de Pruebas (Testing Suite)

5.1 Pruebas Unitarias Backend (JUnit 5 + Mockito)

```

@ExtendWith(MockitoExtension.class)
class QualityMetricServiceTest {

    @InjectMocks
    private QualityMetricServiceImpl qualityMetricService;

    @Test
    @DisplayName("Debe calcular correctamente el % de Calidad alcanzando la meta del 95%")
    void shouldCalculateQualityPercentageCorrectly() {
        // Given
    }
}

```

```

    long totalSuccessfulCases = 100;
    long totalBugsFound = 4; // (1 - 4/100) * 100 = 96%

    // When
    double qualityPercentage = qualityMetricService.calculateQuality(totalSuccessfulCases,
totalBugsFound);

    // Then
    assertEquals(96.0, qualityPercentage);
    assertTrue(qualityPercentage >= 95.0, "La calidad debe superar el target del 95%");
}
}

```

5.2 Pruebas Unitarias Frontend (Angular Component Test)

```

describe('MetricsDashboardComponent', () => {
  let component: MetricsDashboardComponent;
  let fixture: ComponentFixture<MetricsDashboardComponent>;

  beforeEach(async () => {
    await TestBed.configureTestingModule({
      imports: [MetricsDashboardComponent], // Standalone Component
      providers: [provideHttpClient()]
    }).compileComponents();

    fixture = TestBed.createComponent(MetricsDashboardComponent);
    component = fixture.componentInstance;
  });

  it('debe crear el componente y calcular el estado del Target 95%', () => {
    component.qualityMetric.set(96.5);
    fixture.detectChanges();
    expect(component.isTargetAchieved()).toBeTrue();
  });
});

```

5.3 Pruebas End-to-End Automatizadas (Playwright TypeScript)

```

import { test, expect } from '@playwright/test';

test.describe('Flujo de Gobierno y Registro de Retest en QA Portal', () => {

  test.beforeEach(async ({ page }) => {
    await page.goto('https://qa-metrics-portal.vercel.app/login');
    await page.fill('input[name="username"]', 'admin_qa');
    await page.fill('input[name="password"]', 'Secret123!');
    await page.click('button[type="submit"]');
    await expect(page).toHaveURL('https://qa-metrics-portal.vercel.app/dashboard');
  });

  test('Debe registrar un nuevo Retest para un requerimiento de Fábrica', async ({ page }) => {
    await page.click('text=Ejecuciones');
    await page.click('button:has-text("Agregar Retest")');

    await page.selectOption('select[name="projectType"]', 'FABRICA');
    await page.fill('input[name="jiraId"]', 'JIRA-8892');
    await page.fill('input[name="analyst"]', 'Rafael Alvarado');
    await page.selectOption('select[name="status"]', 'SUCCESSFUL');
  });
});

```

```
await page.click('button:has-text("Guardar Iteración")');

const toast = page.locator('.toast-success');
await expect(toast).toContainText('Retest registrado exitosamente');
});
});
```

6. Plan de Migración de Datos y Recomendaciones de Valor

1. **Proceso ETL Automático:** Implementar un script inicial en Python (`pandas` / `openpyxl`) para leer el archivo Excel actual de ejecuciones y poblar las tablas `test_executions` y `test_execution_runs` despivotando las columnas de Retest N.
2. **Webhooks con Jira API:** Sincronizar automáticamente el campo `jira\_id` cuando una historia de usuario cambie al estado "Ready for QA".
3. **Exportación Automática a PDF/XLSX:** Integración de botones de reporte automatizado quincenal para envío por correo a líderes de proyecto.